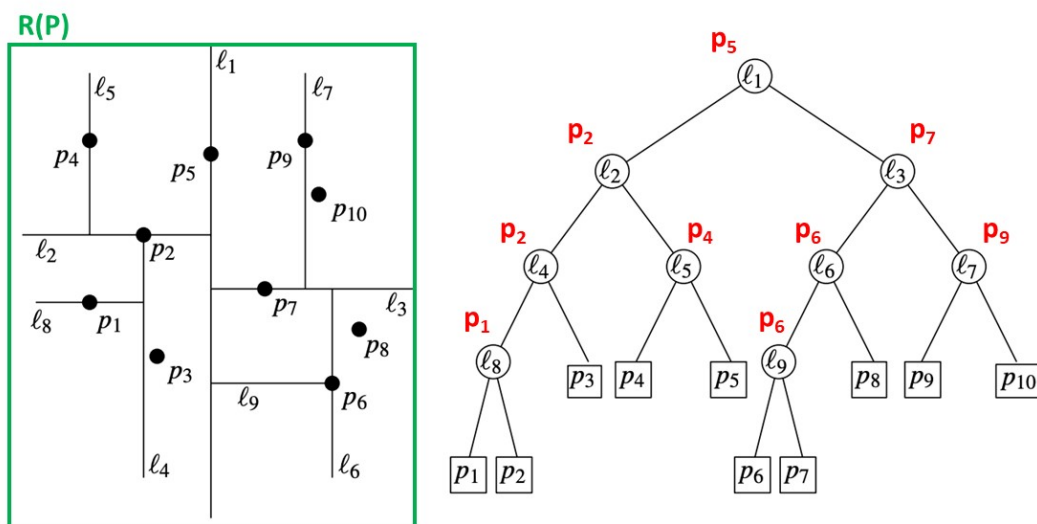


Exercises on Similarity Search

Exercise 1 Let P be a set of n points in $\mathbb{R}^2$ and let T be a kd-tree for P . Suppose that each node v of T stores a point p arbitrarily selected among those associated with leaves of v 's subtree. Show how to adapt `SearchKdTree` so that, given a query rectangle R , it returns an arbitrary point $p \in P \cap R$, if one exists, or `null` otherwise, in time $O(\sqrt{n})$.

Solution. For a node $v \in T$, let $p_v \in P$ denote the point stored in node v . Note that if v is a leaf, its subtree contains only v , hence p_v is uniquely determined, while if v is an internal node, p_v is arbitrarily selected from its subtree's leaves. An example showing a dataset P , the corresponding kd-tree T , and a selection of points stored at the internal nodes, is given in the picture below.



To accomplish the requested task, the idea is to modify `SearchKdTree` so that it stops as soon as it reaches a node v such that $p_v \in R$, returning p_v , and it returns `null` if no such node is ever reached. The pseudocode is the following.

`SearchKdTreeNew( $v, R$ )` // first invocation: $v = T.\text{root}$

Input: A node v in the kd-tree, and a rectangle R

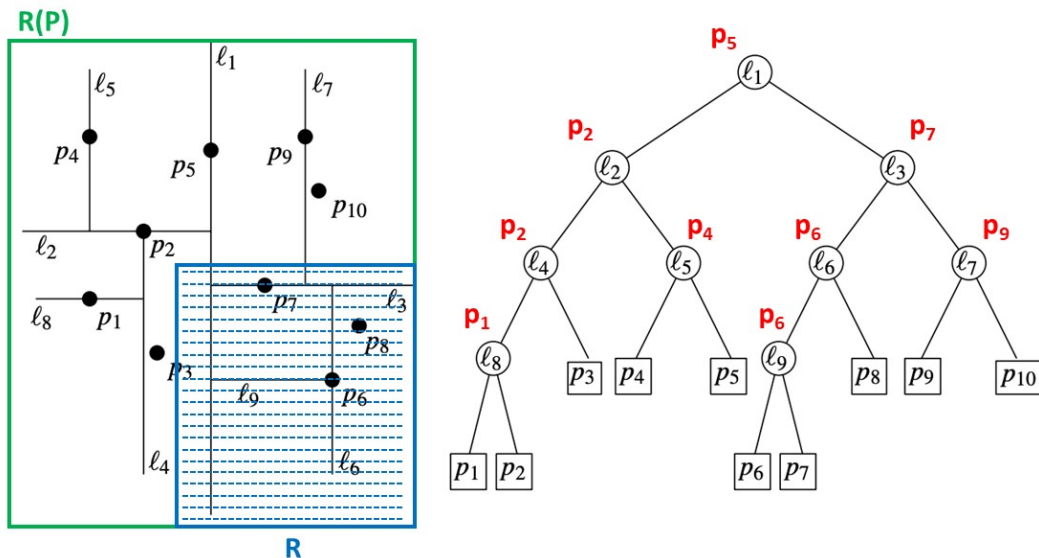
Output: An arbitrary point $p \in R$ from the subtree of v , if one exists, or null otherwise

```

if ( $v$  is a leaf) then
  if ( $p_v \in R$ ) then return  $p_v$ ;
  else return null;
if ( $p_v \in R$ ) then return  $p_v$ ;
else
  Let  $u, w$  be the left and right child of  $v$ , respectively;
  Let  $R_u = \text{region}(u) \cap R$  and  $R_w = \text{region}(w) \cap R$ ;
  if ( $R_u \neq \emptyset$ ) then
     $p \leftarrow \text{SearchKdTreeNew}(u, R_u)$ ;
    if ( $p \neq \text{null}$ ) then return  $p$ ;
  if ( $R_w \neq \emptyset$ ) then return  $\text{SearchKdTreeNew}(w, R_w)$  ;
return null

```

As an exercise, in the example below, show the visited nodes and the output for the given query rectangle R .



The analysis of the query time is similar to the one done in class for `SearchKdTree` (see Slides 28-31 on Similarity Search - Part 1). Suppose that `SearchKdTreeNew` is initially invoked at the root of tree T . We have that the query time is proportional to the number of nodes v on which it is recursively invoked, since in each such invocation performs $O(1)$

operations (excluding the recursive calls). By construction, the algorithm is invoked only on nodes v whose associated region ($\text{region}(v)$) intersects R (except possibly the root). Define:

$$\begin{aligned} Q_1(R) &= \{v \in T : \text{region}(v) \cap R \neq \emptyset \text{ and } \text{region}(v) \not\subseteq R\} \\ Q_2(R) &= \{v \in T : \text{region}(v) \cap R \neq \emptyset \text{ and } \text{region}(v) \subseteq R\} \end{aligned}$$

Recall that in class we claimed $|Q_1(R)| = O(\sqrt{n})$. It is immediate to see that `SearchKdTreeNew`($T.\text{root}, R$) will be recursively invoked on the root and at most all nodes $Q_1(R)$ and one node of $Q_2(R)$. This derives from the fact and if at some point `SearchKdTreeNew` is invoked on a node $v \in Q_2(R)$, then $p_v \in R$, hence the algorithm will return p_v and v will be the last node on which `SearchKdTreeNew` is invoked. The above discussion implies that the running time is $O(\sqrt{n})$. $\square$

Exercise 2 Let P be a collection of n documents that you want to store into a suitable data structure so to retrieve, given a query document q , a similar document in P . Let W be a set of D relevant words and suppose that the similarity between two documents depends on the number of words of W in common (ignoring their relative frequencies).

1. Describe a representation of the documents and a data structure for P based on a suitable locality-sensitive family of hash functions (use only one hash function for the data structure).
2. Based on the above point, find values of c and r such that the (c, r) -ANNS problem for P can be solved correctly with probability at least $1/2$ and expected query time $O(n)$. (Observe that the trivial exact approach requires $O(Dn)$ query time.)

Solution.

1. Let $W = \{w_0, w_1, \dots, w_{D-1}\}$. We can represent a document p as a binary vector V_p , where $V_p[i] = 1$ if p contains w_i , and $V_p[i] = 0$ otherwise, for $0 \leq i < D$. Then, the distance $d(p, q)$ between two documents p, q can be defined to be the Hamming distance between V_p and V_q , that is,

$$d(p, q) \stackrel{\text{def}}{=} d_H(V_p, V_q)$$

We can then use the *bit sampling approach* studied in class for binary vectors. Specifically, we store P in a hash table T constructed by selecting a hash function uniformly at random from the family $\mathcal{H}_H$ defined as follows. For $0 \leq i < D$ let $h_i : P \rightarrow \{0, 1\}$ be the hash function such that $h_i(p) = V_p[i]$, for every $p \in P$, and define

$$\mathcal{H}_H = \{h_i : 0 \leq i < D\}.$$

2. Given two documents p, q , the probability of collision is

$$\Pr_{h \in \mathcal{H}_H} [h(p) = h(q)] = 1 - \frac{d_H(V_p, V_q)}{D} = 1 - \frac{d(p, q)}{D}$$

Therefore,

- if $d(p, q) \leq r$, then

$$\Pr_{h \in \mathcal{H}_H} [h(p) = h(q)] = 1 - \frac{d(p, q)}{D} \geq 1 - \frac{r}{D},$$

- if $d(p, q) \geq cr$, then

$$\Pr_{h \in \mathcal{H}_H} [h(p) = h(q)] = 1 - \frac{d(p, q)}{D} \leq 1 - \frac{cr}{D},$$

which implies that $\mathcal{H}_H$ is (c, r, p_1, p_2) -locality sensitive with $p_1 = 1 - r/D$ and $p_2 = 1 - cr/D$. Now, using the hash table T , we can solve the (c, r) -ANNS problem for a query document q by returning an arbitrary document p in $T[h(q)]$ with $d(p, q) \leq cr$, if one is found, or null otherwise. By the analysis shown in class (see Slides 15-19 on Similarity Search - Part 2) we know that the probability that the output is correct is $\geq p_1$ and the expected query time is $O(Dnp_2)$. So our final goal is to determine the parameters c and r so to make $p_1 = 1/2$ and $p_2 = O(1/D)$. To this purpose, we can fix $r = D/2$ and $c = 2(1 - a/D)$ for some constant $a > 1$ (which implies $cr = D - a$). This yields $p_1 = 1/2$ and $p_2 = a/D$.

□

Exercise 3 Let P be a set of n D -dimensional Boolean vectors. Suppose that P is stored into a hash table T built using a hash function h randomly drawn from the bit-sampling LSH family $\mathcal{H} = \{h_i : 0 \leq i < D\}$. For $h \in \mathcal{H}$, let " $\text{not}(h(x))$ " denote the negation of the binary value $h(x)$.

1. Given two vectors p and q , determine the probability $\Pr_{h \in \mathcal{H}}[h(p) = \text{not}(h(q))]$ as a function of the Hamming distance $d_H(p, q)$.
2. Given a query vector q , we want to find an r -far vector $p \in P$, i.e., such that $d_H(p, q) \geq r$. Based on the above analysis, how would you efficiently search such a p in the table T ? What probabilistic guarantees does your method provide?

Solution.

1. For $0 \leq i < D$, let p_i and q_i denote the i -th bit of p and q , respectively. Clearly,

$$h_i(p) = \text{not}(h_i(q)) \Leftrightarrow p_i \neq q_i.$$

Therefore,

$$\Pr_{h \in \mathcal{H}} [h(p) = \text{not}(h(q))] = \frac{d_H(p, q)}{D}$$

2. Given a query vector q one can scan bucket $T[\text{not}(h(q))]$ returning the first vector p with $d_H(p, q) \geq r$, if one exists, or null otherwise. If there exists an r -far vector $x \in P$, the probability of returning a correct solution is at least the probability that x ends up in bucket $T[\text{not}(h(q))]$. Based on the previous analysis, this probability is

$$\Pr_{h \in \mathcal{H}}[h(x) = \text{not}(h(q))] \geq \frac{r}{D}.$$

□